

C++ Should Be C++

Document number: P3023R0

Date: 2023-10-14

Authors: David Sankel <dsankel@adobe.com (mailto:dsankel@adobe.com)>

Audience: Evolution, Library Evolution

Abstract

Over the past few years, the C++ community has dealt with scandals, calls for a so-called successor, and threats of anti-C++ safety regulations. All this is on top of the ordinary committee stress of competing designs and harsh prioritization. In a time like this it's easy to dwell on our troubles or take fiercely defensive positions.

This paper reframes the unconstructive narratives and argues that the committee's real opportunity is to improve people's lives. We'll show how this outlook leads to guidance on committee participation, direction, and responsibilities.

Introduction

What follows is an outline for a paper I hope to present at the 2023 Kona meeting. My apologies for not having the complete version ready for this mailing, but a family situation made this impossible. Please check for a newer revision for the full treatment.

Mission

The biggest threat

The surest way to sabotage a standard is to say yes to everything.

– Scott Foshee, technology standards veteran

- When you think of threats to C++, what commonly comes to mind? Rust? Safety legislation? Better languages?
- In actuality only one organization in the world holds the power to make C++ less useful: the C++ standardization committee.
- Takeaways:
 - Saying “no” preserves existing value. It should be the default option.

- The bar for making changes should be high. Doubts of whether or not a proposal adds value should cause alarm.
- When value is destroyed, there is little recourse to restore it.
- We should be saying “no” more often.

Mission for the standardization committee

- Counter-productive missions:
 - Make/keep C++ the best language in the world
 - Make C++ the only language people use
 - Make C++ the most popular language
 - Make C++ the best language for a particular task
- Alternatively, we can attempt to improve people’s lives through our work on C++. This applies to developers, users, and society.
- When this mission is adopted, we’ll see that so-called “competitors” are actually allies who are also improving people’s lives.
- Takeaways
 - Turf war mentality hurts our cause
 - Making comparisons saps energy and breeds jealousy
 - We should be happy when our users find solutions to their problems outside of C++.
The important thing is that they found a solution.

The social aspect

C++ as personal and group identity

- People use C++ to define their personal identity and purpose
- Resulting emotional baggage makes reasoned discussion difficult.
- Tribalism creates skepticism of “out-groups”; good ideas are rejected on the basis of where they came from
- Takeaways:
 - Being “part of” C++ is not a vehicle for life fulfillment
 - We need reminders of our mission

Language wars

I can think of few things more depressing than C++ still being used in 1,000,000 years

– Lisa Lippencott

- People frequently use phrases like “death”, “C++ killer”, “language wars”, and other militaristic personifications of programming languages
- These personifications are fundamentally inaccurate
- They also produce counter-productive “fear of death”, and “joy of conquer” emotions
- Language war mentality minimizes the contributions other languages have on our own feature sets
- Takeaways:
 - Languages don’t battle, people do
 - Smearing other languages doesn’t improve people’s lives
 - Longevity of C++ is a non-goal

Standardization as personal opportunity vs. stewardship

- It is easy to fall into a trap thinking standard committee participation is primarily a personal opportunity for you to
 - become an expert in C++,
 - rub shoulders with C++ “celebrities”, or
 - worse of all, leave your mark on the world by getting a proposal accepted.
- There is nothing you have done to earn a role that can impact millions of people’s lives. This should be massively, massively humbling.
- All personal opportunities should be overshadowed by stewardship
 - You are making decisions that will impact millions of people’s lives
 - Every “yes” of yours has the potential to make the world a worse place
- Takeaways:
 - Throw away ideas of leaving a mark on the world
 - Take responsibility for your role. Don’t vote in favor of anything you don’t understand or have questions about.
 - Make informed decisions: Read the paper, reach out to the authors, and ask your questions.
 - You cannot afford to be bullied
 - Enlist the help of experienced designers and wording experts. They want to help!

The technical aspect

Anti-patterns at work

Neophilia

- Just because a language has something better doesn't mean we should incorporate it. Consider traits.
- "Keeping up with the Joneses" is a disservice to our users.
- Frequently C++ additions poorly integrate and burden novices.
- How will non-experts react when they need to jump into this code?

Features and prioritization bias towards experts

- The committee is expert heavy and this is hard to avoid
- This leads to a lopsided number of proposals that primarily target experts
- Average users aren't making use of these features and their needs aren't being met
- Esoteric languages have less impact

Complexity

- C++ is a language with a style, pattern, etc.
- Committee size growth leads to forgotten history which leads to deprioritization of consistency
- Onboarding C++ developers is increasing in cost as the standard grows in complexity
- Takeaways
 - Push for coherent whole
 - Consider the entirety of our offering when deciding to add something

Niche problems getting more than niche effort

- We frequently spend time on things that only a small number of people care about
- It's difficult to say no when someone would benefit, but focusing on the masses is a better use of time and resources

Some big topics put in perspective

- *Safety*. Rust is a great solution for safety-critical components. How are we improving people's lives by spending time on adding a less-capable alternative to C++?
- *Cpp2*. New, improved languages are great, but we need an undistracted committee that focuses on using C++ to improve people's lives.
- What should we be doing instead?
 - Faster hashing and hash combinators
 - JSON parsing
 - Command-line library
 - Coroutine support libraries

- etc.